

CSE428 Lecture 7.0: Introduction to Machine Learning

ML Notations & Terminologies

- **Data**

Data is the basic building block of Machine Learning. It is the information from which the model learns patterns.

Example: Photos of cats and dogs, or customer records in a bank.

- **Training**

Training is the process of teaching a machine learning model using data. The model learns by adjusting itself to reduce mistakes.

- A single data item is called a **training example/sample**
- All training data together is called the **training set**

Example: Showing thousands of cat and dog images so the model learns to recognize them.

- **Test Set**

The test set is a separate set of data that the model has never seen before. It is used to check how well the model performs in real situations.

Example: After training on many cat/dog images, we test it with new images it has never seen.

- **Model / Hypothesis**

A model is the final “learned system” that represents how data is related. It is used to make predictions on new data.

Example: A trained model that can look at an image and say “cat” or “dog”.

• Features / Input

Features are the important details taken from data that help the model make decisions.

Example: In an image, features can be shape, color, or texture of a cat or dog.

• Labels / Output

Labels are the correct answers that the model is trying to predict. In supervised learning, the model learns by comparing its output with labels.

- True label = t
- Predicted output = y

Example: Label = “cat” or “dog”.

• Parameters / Weights

Parameters (or weights) are internal values of the model that are learned during training. They control how input features are converted into output.

Example: The model adjusts weights to better distinguish cat ears vs dog ears.

• Learning Algorithm

It is the method used to train the model and update its parameters. It helps reduce errors step by step.

Example: Gradient Descent adjusts the model so it makes fewer mistakes in classifying cats and dogs.

• Overfitting

Overfitting happens when the model learns the training data too well, including noise and unnecessary details. As a result, it performs poorly on new data.

Example: The model memorizes specific cat images but fails to recognize a new cat it has never seen.

Summary

- Data → Information used for learning
- Training → Teaching the model
- Test set → Checking performance
- Model → Final learned system
- Features → Input details
- Labels → Correct answers
- Weights → Internal settings
- Learning algorithm → Training method
- Overfitting → Memorizing instead of learning

Predicting Price of a Flat (ML Notations & Terminology)

The given dataset is used to build a machine learning model that predicts the **price of a flat** based on its features.

Data (Training Set + Test Set)

	Living Area (feet ²)	#Bedrooms	Price (X1000\$)
	2104	3	400
i^{th} Training Example/Sample $\mathbf{x}^{(i)}, \mathbf{t}^{(i)}$ [here, $i = 2$]	1600	3	330
	2400	3	369
	1416	2	232
	3000	4	516
	•	•	•
	•	•	•

Input/Features, $\mathbf{x}$
Target/Labels, $\mathbf{t}$

Task: Predicting the price of the flat from the given Living area and # of Bedrooms.

Data (Training Set + Test Set)

The dataset contains information about houses/flat:

- Living Area (feet²)
- Number of Bedrooms
- Price (X1000\$)

This whole dataset is called **Data**, which is divided into:

- Training Set → used to train the model
- Test Set → used to evaluate the model

Training Example / Sample ($\mathbf{x}^{(i)}, \mathbf{t}^{(i)}$)

Each row in the table is a training example:

Example from the image:

- $i = 2$
- $\mathbf{x}^{(2)} = (1600, 3)$
- $\mathbf{t}^{(2)} = 330$

So one training sample is:

$(x(i), t(i)) = (\text{Living Area, Bedrooms, Price})$

Input / Features (x)

These are the inputs used to predict price:

- Living Area (feet²)
- Number of Bedrooms

So: $x = (\text{Living Area, Bedrooms})$

Example:

- $x = (2104, 3)$

Target / Labels (t)

This is the output we want to predict:

- Price of the flat (in \$1000)

So: $t = \text{Price}$

Example:

- $t = 400$

Task

The goal of the model is:

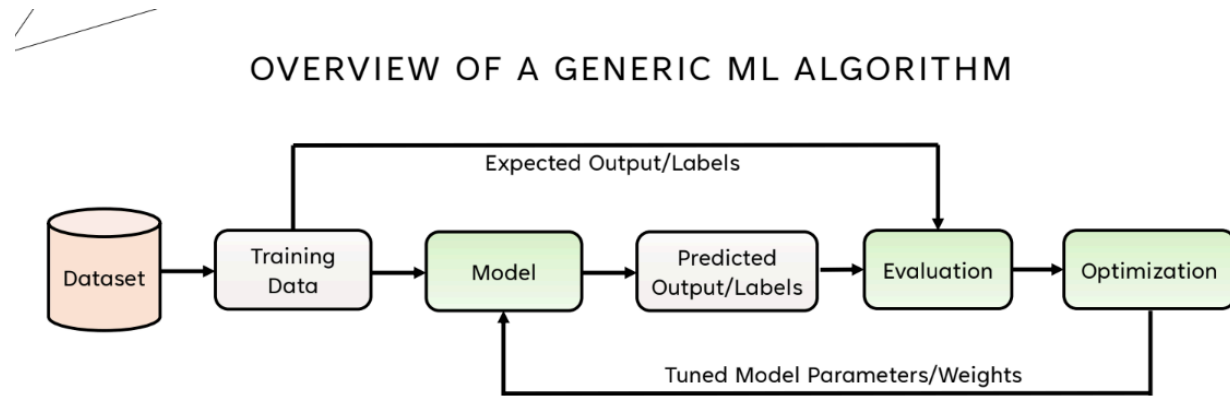
Predict the price of a flat using:

- Living Area
- Number of Bedrooms

So the mapping is: $(\text{Living Area, Bedrooms}) \rightarrow \text{Price}$

Simple Explanation

The machine learning model learns from past house data and then predicts the price of a new flat based on its size and number of rooms.



The Diagram above represents **ML Workflow** in general. An **ML Algorithm** is made up of three elements related to three stages in this workflow. They are:

- **Model:** The mathematical representation of the problem (Structure of the algorithm/model).
- **Evaluation:** Loss Function (the model parameters are tuned to minimize this value).
- **Optimization:** Learning Algorithm (dictates the model tuning process).

A Machine Learning algorithm is mainly built using three important parts:

1. Model (Hypothesis)
2. Learning / Optimization Algorithm
3. Cost Function (Objective Function)

1. MODEL (HYPOTHESIS)

The model is the mathematical structure that represents the problem. It is trained using data to make predictions or decisions.

You can think of it as the “brain” of the system that learns patterns from data.

Different types of models include:

- Linear Regression
- Decision Trees

- Neural Networks

Example:

If we want to predict house prices, the model learns how features like size, location, and rooms affect the price and then predicts the price for new houses.

2. LEARNING / OPTIMIZATION ALGORITHM

This is the method used to improve the model.

It adjusts the model's parameters (called weights) based on training data so that predictions become more accurate.

Common optimization methods include:

- Gradient Descent
- Backpropagation (used in neural networks)
- Other optimization techniques

Example: If the model makes wrong predictions, the learning algorithm slowly changes the weights so that future predictions become better.

3. COST FUNCTION (OBJECTIVE FUNCTION)

The cost function measures how wrong the model's predictions are compared to the actual correct values.

The goal of the learning process is to **minimize this error**.

At the level of a single data point, it is called a **Loss Function**.

Common examples:

- **Mean Squared Error (MSE):** used for regression problems (like predicting house prices)
- **Cross-Entropy Loss:** used for classification problems (like spam vs not spam)

Example: If the actual house price is 100,000 but the model predicts 90,000, the cost function calculates the error between them.

SUMMARY

- **Model:** The system that makes predictions (e.g., neural network, decision tree)
- **Learning Algorithm:** Improves the model using data (e.g., gradient descent)
- **Cost Function:** Measures prediction error and helps the model learn by reducing it

Together, these three components allow a machine learning system to learn from data and improve over time.

OVERVIEW OF A GENERIC ML ALGORITHM (MODEL)

Data (Training Set + Test Set)

	Living Area (feet ²)	#Bedrooms	Price (X1000\$)
	2104	3	400
$x^{(2)}, t^{(2)}$	1600	3	330
	2400	3	369
	1416	2	232
	3000	4	516
	.	.	.
	.	.	.

Input/Features, x Target/Labels, t

Here,

- Training set = $\{(x^{(i)}, t^{(i)})\}$, for $i = 1, 2, \dots, n$,
where, n = number of samples in the training set.

- $x^{(i)} = [x_1^{(i)} \ x_2^{(i)}]$ As, #Features = 2 here.

- So, $x^{(2)} = [1600, 3]$ & $t^{(2)} = 330$ in this case.

- Let's Assume, $y = h_{\theta}(x)$

- Here, $h_{\theta} : x \rightarrow y$ (mapping y from x)

Hypothesis / Model

- Example: Assume, $y = 0.1x_1 + 50x_2 + 12$

θ denotes the **Parameters(Weights)** of the Model

$$\text{Here, } h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i$$

Where, $\theta_1 = 0.1$, $\theta_2 = 50$, $\theta_0 = 12$

OVERVIEW OF A GENERIC ML ALGORITHM (COST FUNCTION)

Data (Training Set + Test Set)

	Living Area (feet ²)	#Bedrooms	Price (X1000\$)
	2104	3	400
$x^{(2)}, t^{(2)}$	1600	3	330
	2400	3	369
	1416	2	232
	3000	4	516
	.	.	.
	.	.	.

Input/Features, x Target/Labels, t

We Assumed, $y = 0.1x_1 + 50x_2 + 12$

$$\text{Here, } h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i$$

Where, $\theta_1 = 0.1$, $\theta_2 = 50$, $\theta_0 = 12$

- In Vector form, $h_{\theta} = \theta^T x$ or, $x^T \theta$
- How good is this model/hypothesis? We need an *Evaluation Metric*. This *evaluation metric* is the **Objective Function** or **Cost Function**, $J(\theta)$.
- The predicted label(price) should be as close as possible to the actual label. So, $y = h_{\theta}(x)$ can be compared to t .
- The *objective* of the ML algorithm will be to minimize the difference (loss) between $h_{\theta}(x)$ & t . Hence, **Minimizing the value of $J(\theta)$** .

Example: Let, $J(\theta) = \frac{1}{n} \sum_{i=1}^n J_i(\theta)$ Where, $J_i(\theta) = [t^{(i)} - h_{\theta}(x^{(i)})]^2$. This is known as MSE (Mean Squared Error).

34

OVERVIEW OF A GENERIC ML ALGORITHM (OPTIMIZING ALGORITHM)

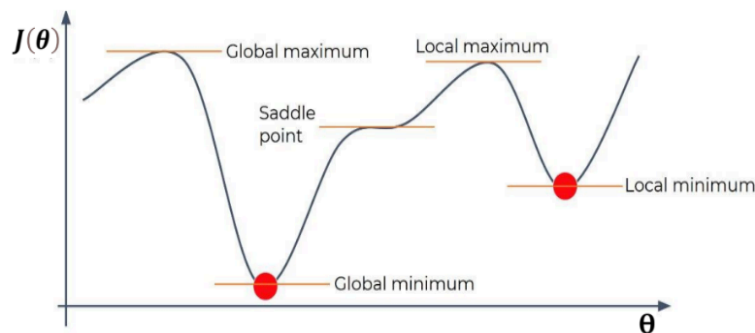
Data (Training Set + Test Set)

	Living Area (feet ²)	#Bedrooms	Price (X1000\$)
	2104	3	400
$x^{(2)}, t^{(2)}$	1600	3	330
	2400	3	369
	1416	2	232
	3000	4	516
	.	.	.
	.	.	.

Input/Features, x
Target/Labels, t

- The *objective* of the ML algorithm will be to minimize the difference (loss) between $h_{\theta}(x)$ & t . Hence, **Minimizing the value of $J(\theta)$** .
- Let, after *evaluation*, some value of $J(\theta)$ was observed. Now, we need some method to keep reducing this value of the Cost function. That's where the **Learning/Optimizing Algorithm** comes in.
- The **Optimizing Algorithm** typically tunes the parameters(weights). So, the purpose of this algorithm is to change $\theta = [\theta_0, \theta_1, \dots, \theta_n]$ in such a way so that $J(\theta)$ is reduced.
- The basic idea is to calculate the *gradient (or slope)* of the loss function with respect to each parameter (weight), and then **update the parameters in the direction that reduces the loss**.

OVERVIEW OF A GENERIC ML ALGORITHM (OPTIMIZING ALGORITHM)



- In the graph above, the x-axis represents the combination(simplified) of the parameters $\theta = [\theta_0, \theta_1, \dots, \theta_n]$ and the y-axis represents the value of the cost function, $J(\theta)$.
- Basically, the *Optimization Algorithm* aims to find the global minima or at least get as close as possible to it, so that the cost is minimal. **To find the combination of the parameters that results in that minimal value, calculating the gradients(derivatives) can be crucial, as they contain information on which direction should the parameters be changed to reach the minima.**
- Many Optimization Algorithms utilize the derivatives due to this fact, in order to find the minima.

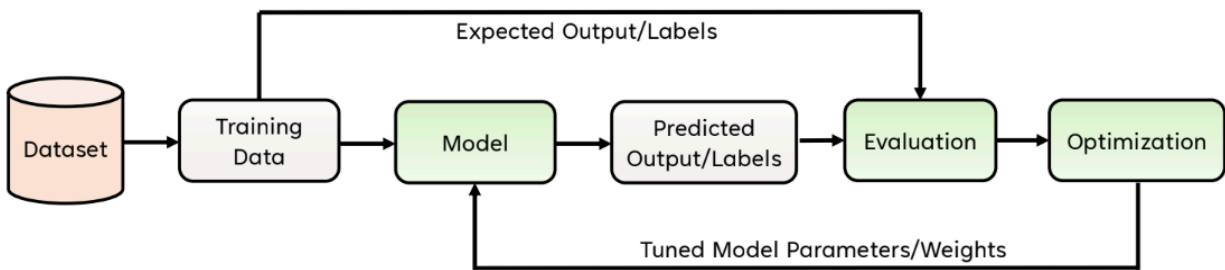
36

CSE428 Lecture 7.1 INTRODUCTION TO SUPERVISED LEARNING & ML ALGORITHMS: LINEAR REGRESSION

Overview of a Generic Supervised Learning Algorithm

This diagram shows the **general workflow of supervised learning**.

Step-by-Step Workflow



1. Dataset

- We start with a dataset containing:
 - Input features (x)
 - Expected outputs/labels (t)

2. Training Data

- The dataset is divided and used as **training data**
- This data is used to teach the model

3. Model

- The model is a **mathematical function** that learns the relationship between:
 - Inputs (features)
 - Outputs (labels)
- It takes input and produces a **predicted output**

4. Predicted Output

- The model generates predictions based on the input data

5. Evaluation

- The predicted output is compared with the **actual (expected) output**
- This tells us how good or bad the model is

6. Optimization

- The model parameters (weights and bias) are adjusted
- The goal is to **reduce the error**
- This process repeats until the model improves

7. Feedback Loop

- The updated (tuned) parameters are sent back to the model
- The model keeps improving through multiple iterations

Three Important Components of Any Supervised Learning Algorithm

1. Model

- Defines the relationship between input features and output labels
- It is the **core part** of any machine learning algorithm

Key Idea:

The model determines how the system learns patterns

2. Cost Function

- Measures how close the model's predictions are to the actual data

Simple Idea:

- Small error → good model
- Large error → poor model

3. Optimization Algorithm

- Decides how to adjust the model parameters
- Its goal is to **minimize the cost (error)**

Example:

- Gradient Descent is a common optimization method

Linear Regression

Linear regression is a basic and important technique in statistics and machine learning.

It is used to **model the relationship between:**

- A **dependent variable** (target/output), and
- One or more **independent variables** (features/inputs)

Objective of Linear Regression

The goal is to find a **straight-line relationship** that can best predict the output from the input.

Key Idea

Linear regression works well **only if the relationship between input and output is approximately linear**. If the data is not linear, this method may not perform well.

Simple Example

- Input (x): number of hours studied
- Output (y): exam score

If more hours studied leads to higher scores in a straight-line pattern, linear regression works well.

Assumptions of Linear Regression

To get good results, linear regression relies on the following assumptions:

1. Linearity

- The relationship between input and output should be **linear (straight-line)**

2. Independence

- Each data point should be **independent of others**
- One observation should not affect another

3. Homoscedasticity

- The **errors (residuals)** should have **constant variance**
- This means the spread of errors should be similar across all input values

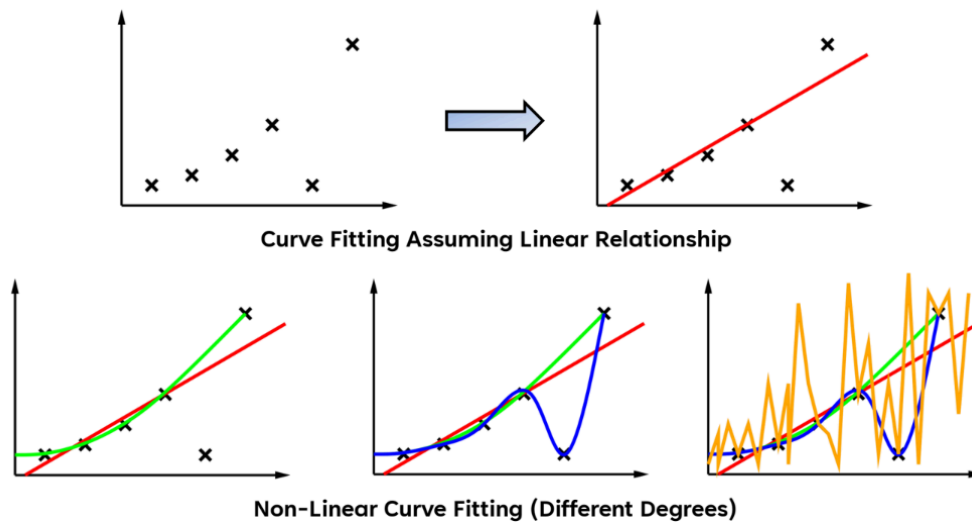
4. Normality

- The **errors (residuals)** should follow a **normal distribution**
- This helps in making reliable statistical conclusions

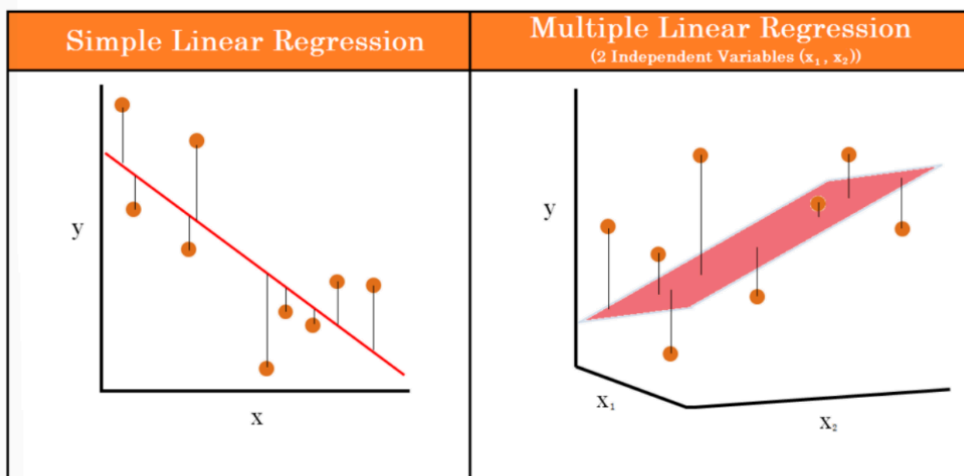
5. No Multicollinearity (for multiple regression)

- The input features should **not be highly correlated with each other**
- If they are strongly related, it becomes difficult for the model to learn properly

ASSUMPTION OF LINEAR RELATIONSHIP



LINEAR REGRESSION IN MULTIPLE DIMENSIONS



Source: <https://medium.com/@thaddeussegura/multiple-linear-regression-in-200-words-data-8bdbcdf34436>

Linear Regression: Model

In linear regression, we use a **linear function** of the input features to predict the output.

Input Representation

- Input features:
 $x = [x_1, x_2, \dots, x_n] \in \mathbb{R}^n$

- Target (actual output):
 $t \in \mathbb{R}$

Model Equation

The prediction is:

$$y = f(\mathbf{x}) = \sum_{j=1}^n w_j x_j + b$$

Where:

- y : predicted value
- (w_j) : weights (importance of each feature)
- b : bias (intercept)
- (n) : number of features

Parameters

- (w) and b together are called **parameters** (Θ)
- We can rewrite the model as:

$$b = \Theta_0, x_0 = 1$$

So the model becomes:

$$y = f(\mathbf{x}) = \sum_{j=0}^n \Theta_j x_j$$

Vector Form

The model can also be written as:

$$y = \Theta^T x = w^T x + b$$

Objective

The goal is: $y \approx t$

That means the predicted value should be **as close as possible** to the actual value.

Linear Regression: Model Example

Dataset

Living Area (ft ²)	Bedrooms	Price (×1000\$)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	516

Features and Target

- (x₁): Living Area
- (x₂): Number of Bedrooms
- Target (t): Price

Model for This Case (n = 2)

$$\begin{aligned}y^{(i)} &= f(\mathbf{x}^{(i)}) = \sum_{j=1}^2 w_j x_j^{(i)} + b \\&= w_1 x_1^{(i)} + w_2 x_2^{(i)} + b\end{aligned}$$

Using parameter form:

$$\begin{aligned}y^{(i)} &= f(\mathbf{x}^{(i)}) = \sum_{j=0}^n \Theta_j x_j^{(i)} \\&= \Theta_1 x_1^{(i)} + \Theta_2 x_2^{(i)} + \Theta_0\end{aligned}$$

Where:

- $x_0^{(i)} = 1$ for all data points
- Parameters: $\Theta = [\Theta_0, \Theta_1, \Theta_2]$

Objective

Find the best values of:

$$\Theta = [\Theta_0, \Theta_1, \Theta_2]$$

So that: $y=f(x)\approx t$

Linear Regression: Cost Function

1. Mean Squared Error (MSE)

The most commonly used cost function is:

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (f(x^{(i)}) - t^{(i)})^2$$

Explanation

- $J(\theta)$: cost (error)
- (n) : number of training examples
- $f(x^{(i)})$: predicted value
- $t^{(i)}$: actual value
- The factor $(1/2)$ is used to make calculations easier

Why MSE?

- It **penalizes large errors more strongly** (because of squaring)
- Creates a **smooth curve**, making optimization easier
- Works well with algorithms like **Gradient Descent**

2. Mean Absolute Error (MAE)

Another cost function is:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n |f(x^{(i)}) - t^{(i)}|$$

Why MAE?

- Less sensitive to **outliers**
- Large errors do not dominate the cost as much as in MSE
- Errors increase in a **linear way**, not squared

Limitation of MAE

- Not differentiable at error = 0
- Makes optimization (like gradient descent) **more difficult or slower**
- Requires special handling or modifications

Linear Regression: Optimization

The main goal in linear regression is to find the **best parameters** (θ) that **minimize the cost function** ($J(\theta)$).

1. Gradient Descent (Iterative Method)

Gradient Descent is a commonly used **optimization algorithm**.

- It improves the model step by step
- In each step, it updates the parameters to **reduce the error**

Update Rule

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} = \theta_j - \frac{\alpha}{m} \sum_{i=1}^m (f(x^{(i)}) - t^{(i)}) \cdot x_j^{(i)}$$

Explanation

- θ_j : parameter being updated
- α : **learning rate** (controls step size)
- m : number of training examples
- $\partial J(\theta) / \partial \theta_j$: gradient (direction of maximum increase in error)

Key Idea

- Move in the **opposite direction of the gradient**
- This reduces the cost function

Convergence

- The algorithm stops when:
 - Changes in $J(\theta)$ become very small
- At this point, the model has found **optimal or near-optimal parameters**

2. Normal Equation (Direct Method)

The Normal Equation is a **direct mathematical solution**.

- It finds the optimal parameters **in one step**
- No iteration is needed

Formula

$$\theta = (X^T X)^{-1} X^T y$$

Explanation

- (X): matrix of input features
- (y): vector of target values
- θ : parameter vector (weights)

Advantages of Normal Equation

- No need to choose a learning rate
- Gives the **exact optimal solution directly**

Disadvantages of Normal Equation

- Computationally expensive
- Requires matrix inversion, which has complexity: $O(n^3)$
- Not suitable for:
 - Large datasets
 - High number of features

Comparison

Method	Type	Key Idea
Gradient Descent	Iterative	Step-by-step error minimization
Normal Equation	Direct	One-step exact solution

GRADIENT DESCENT & ITS VARIATIONS

CALCULUS REVIEW: DERIVATIVES

Definition

A derivative tells us how fast a function is changing when its input changes.

Mathematically, the derivative of a function $f(x)$ with respect to (x) is written as:

- $f'(x)$ or
- $df(x)/dx$

The derivative at a specific point gives the **slope of the tangent line** at that point on the graph.

Example

If: $f(x)=x^2$

Then: $f'(x)=2x$

This means the slope of the curve (x^2) at any point (x) is $(2x)$.

Significance in Machine Learning

Derivatives are used to find out whether a function is increasing or decreasing.

This is very important in **optimization**, where we try to find the **minimum or maximum** of a cost function.

By calculating the derivative of the cost function, we can find the **direction of steepest decrease**, which helps us adjust parameters to reduce the cost.

CALCULUS REVIEW: PARTIAL DERIVATIVES

Definition

Partial derivatives are used when a function has more than one variable.

If a function depends on x and y , written as $f(x, y)$, then:

- Partial derivative with respect to (x) is written as $\partial f(x,y)/\partial x$
- Partial derivative with respect to (y) is written as $\partial f(x,y)/\partial y$

To find a partial derivative, we change only one variable and keep the others constant.

Example

If: $f(x, y) = x^2 + y^2$

Then:

$$\frac{\partial f}{\partial x} = 2x$$

$$\frac{\partial f}{\partial y} = 2y$$

Significance in Machine Learning

Machine learning models often have many parameters. Partial derivatives show how the cost function changes with respect to each parameter. This is important during training because each parameter must be updated in the direction that reduces the cost.

So, partial derivatives tell us **how to adjust each parameter correctly**.

CALCULUS REVIEW: GRADIENTS

Definition

A gradient is a vector that contains all partial derivatives of a function.

For a function $f(x, y)$, the gradient is:

$$\nabla f(x, y) = \left(\frac{\partial f(x, y)}{\partial x}, \frac{\partial f(x, y)}{\partial y} \right)$$

The gradient points in the direction where the function increases the fastest (this is called the **steepest ascent**).

Example

If: $f(x, y) = x^2 + y^2$

Then: $\nabla f(x, y) = (2x, 2y)$

Significance in Machine Learning

In optimization, especially **Gradient Descent**, we use the **negative of the gradient**.

- The gradient shows the direction of fastest increase
- The negative gradient shows the direction of fastest decrease

This helps us update parameters in the best way to reduce the cost function. So, gradients give the most efficient direction for optimization.

CALCULUS: WHY IT MATTERS IN MACHINE LEARNING

1. Finding Minima/Maxima

Optimization in machine learning means finding the **minimum value of a cost function**, which gives the best model parameters.

Derivatives and gradients help find the direction where the cost function decreases fastest.

At the minimum or maximum point: slope (derivative) = 0

2. Efficiency

Knowing the gradient helps us update parameters efficiently. Instead of guessing or trying random directions, the gradient shows the exact direction to improve the model.

3. Convergence

Using derivatives and gradients helps optimization algorithms like **Gradient Descent** reach the best solution. This works even in very complex problems with many variables (high-dimensional spaces).

THE OPTIMIZATION PROBLEM

In **Linear Regression** and most machine learning algorithms, the main goal is to find the best values of parameters:

$$\Theta = [\Theta_0, \Theta_1, \Theta_2, \dots]$$

These parameters are chosen so that they **minimize the cost function** ($J(\theta)$).

There are **two main approaches** to solve this problem:

1. Direct Solution

In this method, we take the derivative of the cost function and set it equal to zero:

$$\frac{\partial J(\theta)}{\partial \theta} = 0$$

Then we solve this equation to find the best values of (Θ).

Idea

At the best point (minimum or maximum), the slope becomes zero.

Problem

- This method is not always possible
- It becomes very expensive (slow and heavy computation) when:
 - dataset is very large
 - there are many features

So, it is not practical for most real-world machine learning problems.

2. Iterative Algorithm

In this method, we improve the parameters step by step instead of solving directly.

Definition

An iterative algorithm means:

- we start with initial values
- then repeatedly update them
- until a stopping condition is met

Process

1. Initialize weights (for example, all zeros or small values)
2. Compute how to improve them
3. Update them in the direction of **steepest descent**
4. Repeat until the cost becomes very small or stops improving

This process is called: Gradient Descent

Idea

Instead of jumping directly to the answer, we slowly move toward the best solution step by step.

MODEL INITIALIZATION

Definition

Model initialization means setting the **starting values** of a machine learning model's parameters:

- weights
- biases

This happens **before training starts**.

Good initialization is important because it affects:

- how fast the model learns
- whether it reaches a good solution

Types of Initialization Techniques

1. Zero Initialization

All weights are set to zero.

- Simple to use
- Not recommended for most models (especially neural networks)

Problem

If all weights are zero, all neurons learn the same thing, which makes learning ineffective.

2. Random Initialization

Weights are set to small random values.

- Values are usually taken from:
 - uniform distribution
 - normal distribution
- Common in:
 - linear models
 - neural networks

This helps break symmetry between neurons.

3. Xavier Initialization

Weights are initialized using a scaled random distribution.

- Designed for deep neural networks
- Works well with activation functions like:
 - sigmoid
 - tanh

It helps keep the signal neither too small nor too large while passing through layers.

4. He Initialization

Similar to Xavier initialization but scaled differently.

- Specifically designed for **ReLU (Rectified Linear Unit)** activation function
- Works better in deep networks using ReLU

It helps maintain stable learning in deep layers.

5. Pre-trained Initialization

Instead of starting from scratch:

- we use weights from a model already trained on a related task

This is commonly used in **Transfer Learning**.

Benefit

- Faster training
- Better performance with less data
- Uses knowledge from previous models

GRADIENT DESCENT

Gradient Descent is an **iterative optimization algorithm** used to find the **minimum of a cost function**.

The **cost function** tells us how well a machine learning model is performing (how close predictions are to actual values).

The main goal is:

- Adjust the model's **parameters (weights)**
- So that the **cost function becomes as small as possible**

How Gradient Descent Works

The process has **two main steps**, repeated again and again:

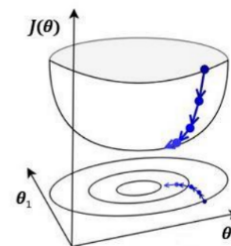
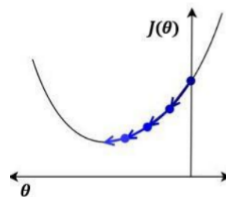
1. Compute the Gradient

- Calculate the derivative (or partial derivatives)
- This tells us the **direction where the function decreases the fastest**

2. Take a Step

- Move the parameters in that direction
- This step helps reduce the cost

Steps Towards the Minima (Where the value of $J(\theta)$ is minimum).



Understanding the Slope

For a parameter (θ_j):

Case 1: Positive Slope

$$\frac{\partial J(\theta)}{\partial \theta_j} > 0$$

- Increasing (θ_j) will **increase the cost function**
- So, we should **decrease (θ_j)**

Case 2: Negative Slope

$$\frac{\partial J(\theta)}{\partial \theta_j} < 0$$

- Increasing (θ_j) will **decrease the cost function**
- So, we should **increase (θ_j)**

Update Rule (Core Formula)

Gradient Descent updates parameters using:

Gradient Descent updates parameters using:

$$\theta_j^{new} = \theta_j^{old} - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

Explanation

- θ_j^{old} : current value
- θ_j^{new} : updated value
- α : learning rate
- $\frac{\partial J(\theta)}{\partial \theta_j}$: slope (gradient)

This update:

- Moves in the **opposite direction of the gradient**
- Helps **reduce the cost function**

Important Note

- This update **always decreases the cost function** if the learning rate is small enough
- If:

$$\frac{\partial J(\theta)}{\partial \theta_j} = 0$$

then:

- No change happens
- This point is a **minimum or maximum**

Learning Rate (α)

Definition

The learning rate α is a **hyperparameter** that controls how big each step is.

- $\alpha > 0$ always
- It decides how fast parameters are updated

Effects of Learning Rate

- **Large α**
 - Faster updates
 - May skip the minimum or become unstable
- **Small α**
 - Slower updates
 - More stable, but takes longer

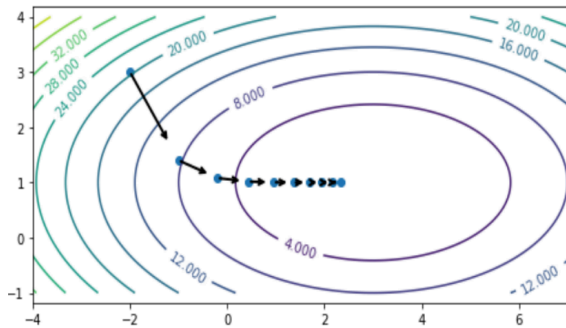
Typical Values

- Usually small numbers like:
 - 0.01
 - 0.0001

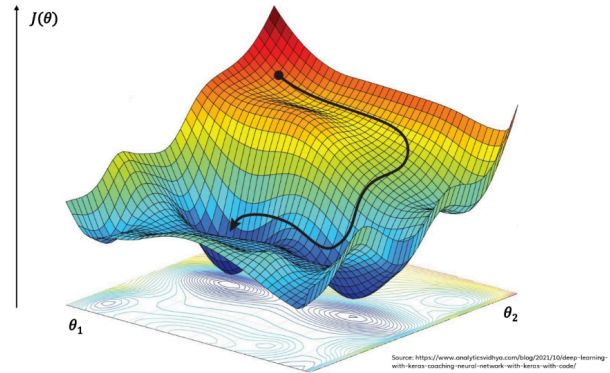
Important Note

- Choosing the right learning rate is important
- It often needs **tuning** for best performance

GRADIENT DESCENT

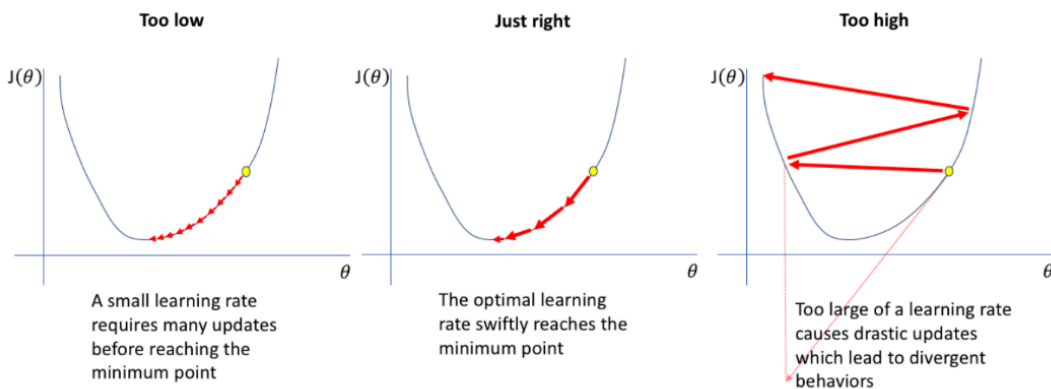


GRADIENT DESCENT (HIGHER DIMENSIONS)

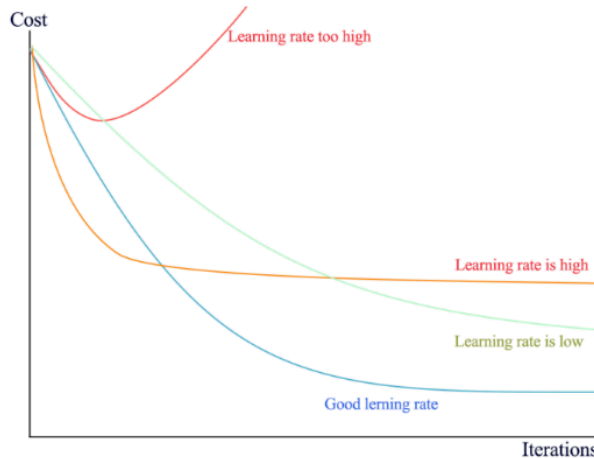


GRADIENT DESCENT: LEARNING RATE

Selecting the optimal learning rate α is critical and often involves experimentation and fine-tuning. Here's what happens with different learning rates:



The value of the Learning Rate α affects the training process & convergence heavily, as we can see below. Techniques such as manual tuning, learning rate schedules, and adaptive learning rates can help in finding an effective learning rate for a given problem.



BATCH GRADIENT DESCENT

Description

In Batch Gradient Descent, we use the **entire training dataset** to compute the gradient.

- We calculate the error for **all training examples**
- Then we **sum and average** these errors
- After that, we update the parameters **once per iteration**

Advantages

- Converges to the **global minimum** if the cost function is convex
- Updates are **stable** because they are based on the whole dataset

Disadvantages

- **Slow for large datasets** because every update uses all data
- Requires **high memory**, since the whole dataset must be loaded

STOCHASTIC GRADIENT DESCENT (SGD)

Description

In Stochastic Gradient Descent (SGD), we update parameters using **only one training example at a time**.

- Instead of waiting for the whole dataset
- We update parameters **immediately after each example**

Advantages

- **Faster** for large datasets
- Uses **less memory**
- Noisy updates help it **escape local minima**

Disadvantages

- **Less stable** (cost function may go up and down instead of smoothly decreasing)
- Cannot fully use **vectorization and matrix operations**, making it less efficient in some implementations

MINI-BATCH GRADIENT DESCENT

Description

Mini-Batch Gradient Descent is a **middle approach** between Batch and SGD.

- Data is divided into small groups called **mini-batches**
- The gradient is computed using **one mini-batch at a time**
- Parameters are updated after each mini-batch

Advantages

- Balances **speed (like SGD)** and **stability (like Batch GD)**

- Can use **matrix operations**, making it efficient for parallel computation

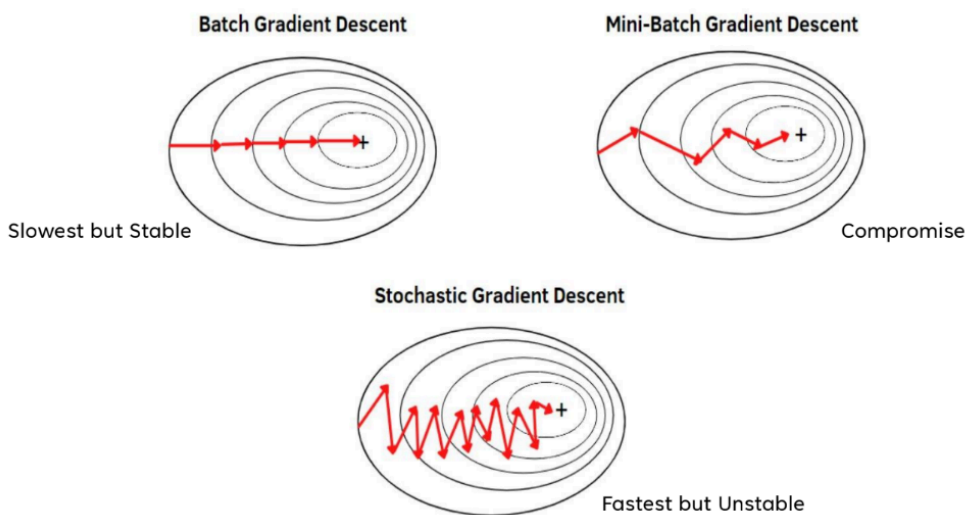
Disadvantages

- Need to choose the right **mini-batch size** (tuning required)
- Updates can still be **slightly noisy**, but less than SGD

Quick Comparison

Method	Data Used per Update	Speed	Stability
Batch GD	Whole dataset	Slow	Very stable
SGD	One example	Very fast	Noisy/unstable
Mini-Batch	Small subset	Balanced	Moderately stable

BATCH VS SGD VS MINI-BATCH



LINEAR REGRESSION: CONVERGENCE

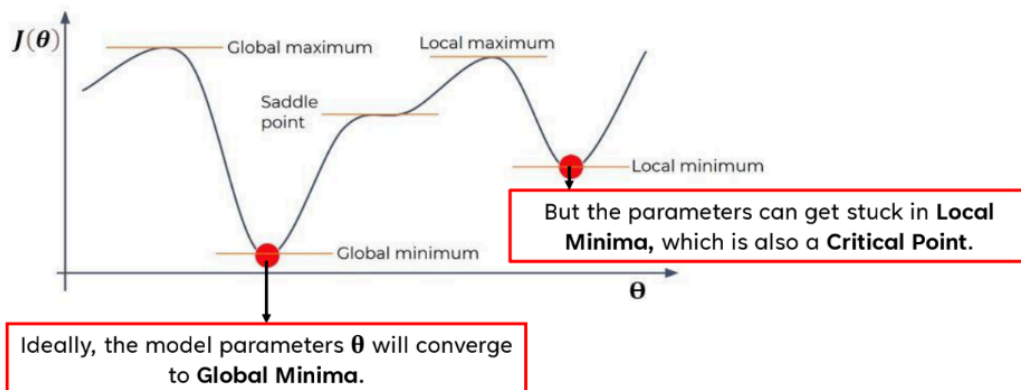
Definition

An optimization algorithm is said to **converge** when:

- Further updates to parameters (θ)
- Cause **little or no change** in the cost function ($J(\theta)$)

At this point:

- The model has (**ideally, but not always**) found the best parameters
- These parameters minimize the **prediction error**



EXAMPLE 1: SINGLE FEATURE (LINEAR REGRESSION WITH GRADIENT DESCENT)

Let's solve a simple linear regression problem analytically using Gradient Descent with just 2 iterations. We'll predict house prices based on the area (single feature).

Problem Setup:

- Feature (X): Area of the house in square feet.
- Target (t): Price of the house in \$1000s.

Dataset

Living Area (sq ft)	Price (in \$1000s)
1000	300
1500	450
2000	500

We want to **predict house prices** using **area (one feature)**.

Step 1: Model Selection

We use **Linear Regression**.

For one feature, the model is:

$$y^{(i)} = \Theta_1 X^{(i)} + \Theta_0$$

Step 2: Cost Function (MSE)

We use **Mean Squared Error (MSE)**:

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (f(X^{(i)}) - t^{(i)})^2$$

Here, ($n = 3$), so:

$$J(\theta) = \frac{1}{6} \sum_{i=1}^3 ((\Theta_1 X^{(i)} + \Theta_0) - t^{(i)})^2$$

Step 3: Initialization

- $\Theta_0 = 0$
- $\Theta_1 = 0$
- Learning rate $\alpha = 0.0000001$

Step 4: Optimization using Gradient Descent

Update rules:

$$\theta_0 = \theta_0 - \alpha \frac{\partial J(\theta)}{\partial \theta_0}, \quad \theta_1 = \theta_1 - \alpha \frac{\partial J(\theta)}{\partial \theta_1}$$

Gradients:

$$\frac{\partial J}{\partial \Theta_0} = \frac{1}{3} \sum_{i=1}^3 ((\Theta_1 X^{(i)} + \Theta_0) - t^{(i)})$$

$$\frac{\partial J}{\partial \Theta_1} = \frac{1}{3} \sum_{i=1}^3 ((\Theta_1 X^{(i)} + \Theta_0) - t^{(i)}) \cdot X^{(i)}$$

Iteration 1

Step 4.1: Predictions

$$y_{pred} = 0 \times X + 0 = 0$$

Step 4.2: Compute Gradients

Gradient for Θ_0 :

$$\frac{1}{3}[(0 - 300) + (0 - 450) + (0 - 500)] = \frac{-1250}{3} = -416.33$$

Gradient for Θ_1 :

$$\begin{aligned} \frac{1}{3}[(-300 \times 1000) + (-450 \times 1500) + (-500 \times 2000)] \\ = \frac{-1975000}{3} = -658333.33 \end{aligned}$$

Step 4.3: Update Parameters

$$\Theta_1 = 0 - (0.0000001 \times -658333.33) = 0.0658333$$

$$\Theta_0 = 0 - (0.0000001 \times -416.67) = 0.00004167$$

Iteration 2

Step 4.1: Predictions

$$y_{pred} = 0.0658333 \times X + 0.00004167$$

- For 1000 $\rightarrow \approx 65.83$
- For 1500 $\rightarrow \approx 98.75$
- For 2000 $\rightarrow \approx 131.67$

Step 4.2: Compute Gradients

Gradient for Θ_0 :

$$\frac{1}{3}[(65.8333 - 300) + (98.67 - 450) + (131.67 - 500)] = \frac{-1417.7533}{3}$$

Gradient for Θ_1 :

$$\begin{aligned} \frac{1}{3}[(-234.1667 \times 1000) + (-351.33 \times 1500) + (-368.33 \times 2000)] \\ = \frac{-1389166.7}{3} = -463055.56 \end{aligned}$$

Step 4.3: Update Parameters

$$\Theta_1 = 0.0658333 - (0.0000001 \times -463055.56) = 0.1121389$$

$$\Theta_0 = 0.00004167 - (0.0000001 \times -472.584) = 0.000088$$

After 2 Iterations

Final parameters:

- $\Theta_1 = 0.1121389$
- $\Theta_0 = 0.000088$

Model:

$$y_{pred} = 0.1121389 \times X + 0.000088$$

Predictions

- $1000 \rightarrow \approx 112.14$
- $1500 \rightarrow \approx 168.21$
- $2000 \rightarrow \approx 224.28$

Step 5: Model Evaluation (MSE)

$$\begin{aligned}
 MSE &= \frac{1}{6}[(112.14 - 300)^2 + (168.21 - 450)^2 + (224.28 - 500)^2] \\
 &= \frac{1}{6}(35283.8596 + 79390.8041 + 76031.3184) \\
 &\approx 31784.5
 \end{aligned}$$

Interpretation

- The model still predicts **much lower values than actual prices**
- But predictions are **better than initial values (which were 0)**
- This shows the model is **learning and improving**

More iterations will:

- Improve predictions
- Move parameters closer to optimal values

Step 6: Model Tuning

If performance is not good, we improve the model by tuning hyperparameters:

Possible Changes

- Learning Rate α
- Number of iterations
- Initialization method
- Optimization algorithm (e.g., SGD instead of basic Gradient Descent)

Key Idea

Even though results are not perfect yet:

- The model is moving in the **correct direction**
- Gradient Descent is **working properly**
- More training = better accuracy

EXAMPLE 2: MULTIPLE FEATURES (GRADIENT DESCENT)

Dataset

Area (X ₁)	Bedrooms (X ₂)	Price (t)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	516

Step 1: Model

For multiple features:

$$y = \Theta_1 X_1 + \Theta_2 X_2 + \Theta_0$$

Step 2: Cost Function

$$J(\theta) = \frac{1}{2n} \sum (y^{(i)} - t^{(i)})^2$$

Here, (n = 5)

Step 3: Initialization

- $\Theta_0 = 0$
- $\Theta_1 = 0$
- $\Theta_2 = 0$
- Learning rate $\alpha = 0.00000001$ (very small due to large values)

Step 4: Gradient Descent Updates

Update rules:

$$\Theta_j = \Theta_j - \alpha \frac{\partial J}{\partial \Theta_j}$$

Gradients:

$$\frac{\partial J}{\partial \Theta_0} = \frac{1}{5} \sum (y - t)$$

$$\frac{\partial J}{\partial \Theta_1} = \frac{1}{5} \sum (y - t) \cdot X_1$$

$$\frac{\partial J}{\partial \Theta_2} = \frac{1}{5} \sum (y - t) \cdot X_2$$

Iteration 1

Step 4.1: Predictions

Since all $\Theta = 0$:

$$y_{pred} = 0$$

Step 4.2: Compute Errors

$$(0-400),(0-330),(0-369),(0-232),(0-516)=-400,-330,-369,-232,-516$$

Step 4.3: Compute Gradients

For Θ_0

$$\frac{1}{5}(-400 - 330 - 369 - 232 - 516) = \frac{-1847}{5} = -369.4$$

For Θ_1

$$\begin{aligned} \frac{1}{5}[(-400 \times 2104) + (-330 \times 1600) + (-369 \times 2400) + (-232 \times 1416) \\ = \frac{1}{5}(-841600 - 528000 - 885600 - 328512 - 1548000) \\ = \frac{-4131712}{5} = -826342.4 \end{aligned}$$

For Θ_2

$$\begin{aligned} \frac{1}{5}[(-400 \times 3) + (-330 \times 3) + (-369 \times 3) + (-232 \times 2) + (-516 \times 4) \\ = \frac{1}{5}(-1200 - 990 - 1107 - 464 - 2064) \\ = \frac{-5825}{5} = -1165 \end{aligned}$$

Step 4.4: Update Parameters

$$\Theta_0 = 0 - (1e-8)(-369.4) = 0.000003694$$

$$\Theta_1 = 0 - (1e-8)(-826342.4) = 0.008263424$$

$$\Theta_2 = 0 - (1e-8)(-1165) = 0.00001165$$

Iteration 2

Step 4.1: Predictions

$$y = 0.008263424X_1 + 0.00001165X_2 + 0.000003694$$

Now compute:

- For (2104,3):
 $y \approx 17.39$
- For (1600,3):
 $y \approx 13.22$
- For (2400,3):
 $y \approx 19.83$
- For (1416,2):
 $y \approx 11.70$
- For (3000,4):
 $y \approx 24.79$

Step 4.2: Errors

$$\begin{aligned} & (17.39 - 400), (13.22 - 330), (19.83 - 369), (11.70 - 232), (24.79 - 51) \\ & = -382.61, -316.78, -349.17, -220.30, -491.21 \end{aligned}$$

Step 4.3: Gradients (Approx)

Θ_0 :

$$\frac{1}{5}(-1760.07) = -352.01$$

Θ_1 :

$$\approx -788000(\text{approx})$$

Θ_2 :

$$\approx -1110(\text{approx})$$

Step 4.4: Update Parameters

$$\Theta_0 = 0.000003694 + 0.00000352 = 0.000007214$$

$$\Theta_1 = 0.008263424 + 0.00788 = 0.01614$$

$$\Theta_2 = 0.00001165 + 0.0000111 = 0.00002275$$

After 2 Iterations

Parameters

- $\Theta_0 \approx 0.0000072$
- $\Theta_1 \approx 0.01614$
- $\Theta_2 \approx 0.00002275$

Prediction for New House

Given:

- Area = 2200
- Bedrooms = 4

$$y = 0.01614(2200) + 0.00002275(4) + 0.0000072$$

$$y \approx 35.51$$

Final Result

$$35.51 \times 1000 = 35,510$$

Important Observation

- Prediction is **very low** compared to actual values
- This is because:
 - Only **2 iterations** were done
 - Learning rate is **very small**

FEATURE MAPPING & POLYNOMIAL REGRESSION

LIMITATIONS OF LINEAR REGRESSION

1. Linearity Assumption

- Linear Regression assumes a **straight-line relationship** between features and target
- But real-world data is often **non-linear (curved or complex)**
- So, the model may fail to capture real patterns

2. Outlier Sensitivity

- Outliers = very large or very small unusual values
- Linear Regression is **highly sensitive** to them
- Even one outlier can **distort the whole model**

3. Independence Assumption

- Assumes all data points are **independent** of each other
- If this is not true (e.g., time-series data), results become **unreliable**

4. Multicollinearity

- Happens when features are **highly correlated** with each other
- Makes the model:
 - unstable
 - hard to interpret

5. No Feature Selection

- The model does **not automatically choose important features**
- Irrelevant features can:
 - increase complexity
 - cause **overfitting**

Key Idea

Some of these problems can be improved using **Feature Mapping**.

FEATURE MAPPING

Feature mapping means **transforming original features into new features**.

The goal is:

- Help the model learn **more complex patterns**
- Even when the model itself is simple (like linear regression)

Why Transform Features?

- Original model assumes **linear relationship**
- After transformation → model can learn **non-linear patterns**

Example

If original feature is x , we can create:

- $(x^2), (x^3)$
- $\sin(x), (e^x)$

This helps the model:

- fit **curves**
- capture **complex relationships**

KEY ASPECTS OF FEATURE MAPPING

1. Higher-Dimensional Spaces (Polynomial Features)

- Create features like:
 - $(x^2, x^3, \dots)$
- This increases dimensions and allows:

- modeling **curved relationships**

Example

Instead of: $y = mx + c$

We get: $y = ax^2 + bx + c$

2. Non-Linear Transformations

Apply functions like:

- $\log(x)$
- $\sin(x)$
- e^x

Useful when relationships are:

- non-linear
- not polynomial

3. Interaction Features

Combine features together.

Example

For (x_1) and (x_2) :

$x_1 \times x_2$

This captures how features **affect each other together**.

4. Kernel Methods

Used in models like **Support Vector Machines (SVMs)**.

- Map data into higher dimensions **without explicitly computing it**
- Allows handling **non-linear data**

5. Encoding Categorical Variables

Convert categories into numbers.

Examples:

- One-hot encoding
- Target encoding

This makes data usable for ML models.

6. Dimensionality Reduction

Reduce the number of features while keeping important information.

Example:

- **PCA (Principal Component Analysis)**
- Converts features into:
 - fewer
 - uncorrelated variables

CHALLENGES OF FEATURE MAPPING

1. Overfitting

- Too many features → model becomes too complex
- Learns noise instead of real pattern

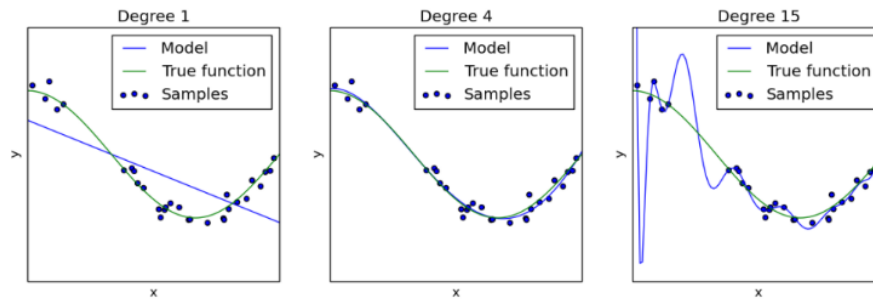
2. Increased Complexity

- More features = harder to:
 - understand
 - compute

3. Feature Selection

- Not all new features are useful
- Need to:

- select important ones
- or use regularization



POLYNOMIAL REGRESSION

Polynomial Regression is a type of **feature mapping** where we add polynomial terms.

It extends linear regression to handle **non-linear relationships**.

Model (Single Feature)

$$y = \Theta_0 + \Theta_1 x + \Theta_2 x^2 + \dots + \Theta_M x^M$$

- (M) = degree of polynomial

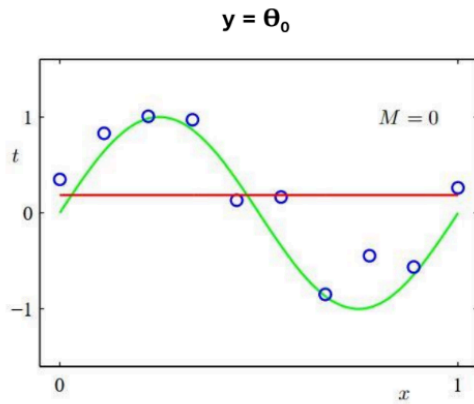
Example (Multiple Features, Degree = 2)

For features (x_1) and (x_2):

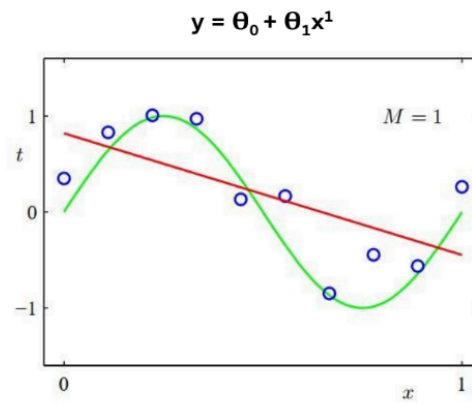
$$y = \Theta_0 + \Theta_1 x_1 + \Theta_2 x_2 + \Theta_3 x_1^2 + \Theta_4 x_2^2 + \Theta_5 x_1 x_2$$

Note

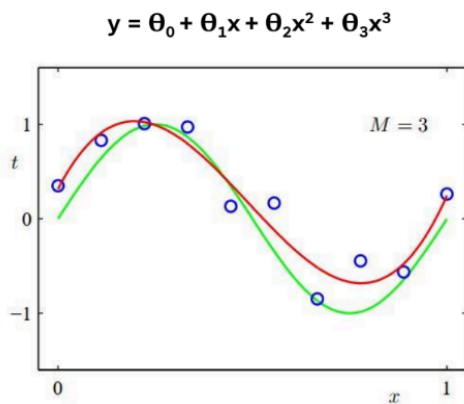
- ($x_1 x_2$) is also degree 2
- So it is included as an **interaction term**



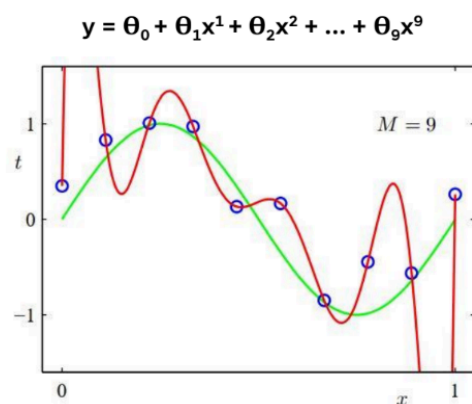
Just a Constant



Model Too Simple
(Underfitting)



Good Fit



Model Too Complex
(Overfitting)

Note

- Linear Regression → simple but limited
- Feature Mapping → makes it powerful
- Polynomial Regression → allows **curved fitting**

Prepared By Azmari Sultana ✨